# Agentic Coding for Economists

Padova five-hour workshop

Antonio Mele

University of Padova, 25 June 2026

Participant repo: `https://github.com/meleantonio/AC4E_Padova`

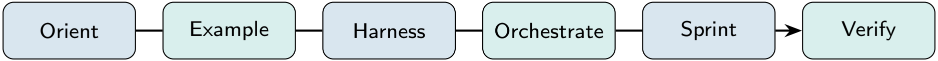
# How To Use This Session

**Pattern**

Short explanation, bounded demo, participant action, verification note.

- Keep `START_HERE.md`, `SCHEDULE.md`, and `GUIDE.md` open.
- Work from one tool lane: Codex, Claude Code, or Cursor.
- Treat agents as implementation assistants, not research owners.
- Every delegated task ends with evidence: diff, command output, PR, or review log.

# Five-Hour Arc




- 00:00-00:45: orientation, safety, agent loop.
- 00:45-01:20: Card-Krueger scaffold.
- 01:30-02:15: skills, subagents, hooks, MCP.
- 02:15-03:00: issues, cloud agents, swarms, loops, goals.
- 03:00-05:00: sprint, verification, adoption.

# Participant Repository Path

**Read first**

- `README.md`
- `START_HERE.md`
- `SCHEDULE.md`
- `GUIDE.md`
- `docs/sources.md`

**Use during demos**

- `examples/card-krueger/`
- `examples/starter_article/`
- `materials/tool_tracks.md`
- `templates/`
- `slides/workshop/demo_map.md`

# What You Produce

- A scoped project brief or design memo.
- Agent instructions and privacy boundaries.
- One usable harness piece: skill, subagent, hook, MCP config, goal, or loop.
- One GitHub issue or cloud-agent task with acceptance criteria.
- One verification artifact: test output, review log, table check, or screenshot.
- A seven-day adoption plan for one real research project.

# Safety Boundaries

**Human-owned decisions**

Research question, identification, data rights, confidentiality, interpretation, and final claims stay with the researcher.

- Do not upload confidential referee material, private data, API keys, or student records.
- Use synthetic or public examples for workshop demos.
- Start read-only; allow edits only after scope is explicit.
- Require commands and outputs before accepting generated code or claims.

# First Read-Only Prompt

## Demo: orientation without edits

```
Read README.md, START_HERE.md, SCHEDULE.md, GUIDE.md,
examples/card-krueger/README.md, and
examples/starter_article/README.md. Do not edit files.

Explain:
1. the workshop path;
2. the Card-Krueger running example;
3. which files I should copy or personalize;
4. which files or folders I should not expose to agents.
```

# Running Case: Card And Krueger

**Question**

What happened to fast-food employment in New Jersey relative to eastern Pennsylvania after New Jersey raised its minimum wage in April 1992?

- Treatment group: New Jersey fast-food stores.
- Comparison group: eastern Pennsylvania fast-food stores.
- Timing: before and after the April 1992 policy change.
- Outcome: full-time-equivalent employment.

# Public Sources And Teaching Data

- Public source links live in `docs/sources.md`.
- David Card's page lists the associated public data archive and readme.
- NBER Working Paper 4509 gives the core research context.
- `examples/card-krueger/` uses synthetic teaching data so everyone can run the demo immediately.

**Caveat**

The bundled CSV is not Card and Krueger's raw data and does not support a substantive causal claim.

# Data Map Checklist

### Required fields

- source and access note;
- unit of observation;
- treatment and comparison groups;
- outcome and units;
- timing and sample restrictions;
- transformations and caveats.

### Workshop file

- `examples/card-krueger/`
- `docs/data_source_map.md`
- `templates/data_source_map.md`

# Baseline DID Mechanics

## Teaching estimand

$$(\overline{Y}_{NJ,after} - \overline{Y}_{NJ,before}) - (\overline{Y}_{PA,after} - \overline{Y}_{PA,before})$$

- The script computes group means and the simple before/after comparison.
- Tests check the input file, group labels, sample counts, command path, and caveats.
- This is a workflow demo, not a replication claim.

# Run The Example

## Demo: baseline command

```
python3 -m pip install -r examples/card-krueger/requirements.txt
python3 examples/card-krueger/src/did_analysis.py
python3 -m pytest examples/card-krueger/tests
```

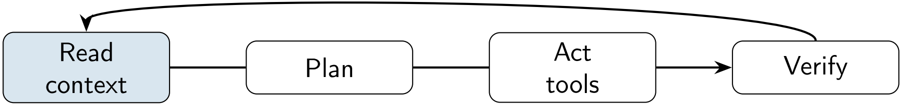
Expected headline:

```
Toy DID estimate: 3.0 FTE workers.
Synthetic teaching data: not Card and Krueger's raw data.
```

# From Example To Your Project

- Use `examples/card-krueger/` as the shared reference during demos.
- Use `examples/starter_article/` as the personalizable harness skeleton.
- Replace the teaching data map with your own data source map.
- Keep the same discipline: scope, evidence, review.

# Agent Loop




- Better context improves plans.
- Better acceptance criteria improves edits.
- Better verification improves trust.

# Harness Pieces

| Piece | Economist use |
|---|---|
| Instructions | persistent project map and boundaries |
| Skills | repeatable procedures for replication, review, writing |
| Subagents | specialist reviewer contexts |
| Hooks | deterministic reminders after edits or before risky commands |
| MCP | structured access to external tools and public data |
| Goals/loops | longer tasks with stop conditions and evidence |

# Project Instructions

- Root `AGENTS.md` tells agents how to work in this repo.
- Project copies can use `AGENTS.md`, `CLAUDE.md`, Cursor rules, or all three.
- Include repo map, allowed files, verification commands, and do-not rules.
- Keep instructions short enough that participants can audit them.

**Handoff:** `templates/AGENTS_economics.md`

# Brief, Memo, Spec

- `docs/project_brief.md`: what the project is and is not.
- `docs/research_design_memo.md`: data, design, risks, and claims.
- `spec/intent.md`: optional spec-driven development path for larger projects.
- Agents can draft; humans own identification and interpretation.

**Handoff:** `templates/project_brief.md`, `templates/research_design_memo.md`

# Skills

**Use when the task is repeatable**

A skill packages a procedure that should be invoked consistently.

- replication checker;
- referee checklist;
- paper polisher;
- research SDD plan.

**Handoff:** `examples/starter_article/.cursor/skills/`

# Subagents And Reviewers

- Use a separate reviewer role for identification, data, literature, or verification.
- Give the reviewer a narrow file set and a blockers-first output format.
- Reviewer agents should not silently fix their own findings.
- In GitHub, request a review after the implementation PR exists.

**Handoff:** `examples/starter_article/.cursor/agents/`

# Hooks

- Hooks are deterministic reminders or commands around agent tool use.
- Example: after editing the DiD script, remind the agent to run pytest.
- Keep hook examples safe and explicit; participants trust them before use.
- If a tool version changes hook behavior, check official docs first.

**Handoff:** `examples/hooks/README.md`

# MCP And Public Data

- MCP gives an agent structured access to tools such as public-data APIs.
- Use read-only public sources first.
- Put secrets in environment variables, never in the repo.
- For Padova, FRED is the shared optional public-data example.

**Handoff:** `examples/cursor/.cursor/mcp.json.example`

# Goals, Loops, And Stop Conditions

- Use a goal for work that spans several steps.
- Define done criteria before starting.
- Define stop conditions: missing data, failed tests, ambiguous identification, tool limits.
- Record each loop in a review or orchestration log.

**Handoff:** `templates/long_running_goal_prompt.md`

# Choose One Lane

| Lane | Tool | Best fit |
|---|---|---|
| A1 | Codex CLI | terminal-first workflows and GitHub tasks |
| A2 | Codex app | project threads, long goals, visual review |
| B1 | Claude Code CLI | terminal coding and explicit subagents |
| B2 | Claude Code app | app-based code sessions |
| C | Cursor | IDE-first coding, rules, background agents |

**Handoff:** `materials/tool_tracks.md`

# Codex Lane

- Use `AGENTS.md` and project-specific instructions.
- CLI and app share the same project files.
- Good for issue-driven implementation, long goals, and PR workflows.
- Verify with commands and attach evidence to PRs.

**Handoff:** `examples/codex/`, `examples/codex-app/`

# Claude Code Lane

- Use `CLAUDE.md` plus shared `AGENTS.md` when collaborating.
- Skills, subagents, and hooks are useful for research review loops.
- Keep project settings in the project, user secrets outside the repo.
- Ask the reviewer role for blockers before fixes.

**Handoff:** `examples/claude/`, `examples/claude-app/`

# Cursor Lane

- Use rules for persistent project context.
- Use IDE review panes and background agents for branch work.
- Skills and subagents can mirror the same economics harness pattern.
- Verify UI labels and hook support against the installed Cursor version.

**Handoff:** `examples/cursor/`

# Version-Sensitive Tool Claims

- Tool surfaces change quickly.
- Do not memorize UI labels from slides.
- Use `docs/sources.md` for official docs.
- If a feature is missing, fall back to files and GitHub issues.

# GitHub As Control Plane

- Issue: task card with scope and acceptance criteria.
- Branch: isolated implementation.
- PR: reviewable diff plus verification evidence.
- Merge: only after evidence and human review.

**Handoff:** `templates/review_log.md`

# Issue Template For Agents

```
Title: Add Card-Krueger robustness check

Allowed files:
- examples/card-krueger/src/did_analysis.py
- examples/card-krueger/tests/test_did_analysis.py

Acceptance criteria:
- baseline output remains unchanged;
- robustness result is documented;
- pytest passes;
- synthetic-data caveat stays visible.
```

# Cloud Agents And PR Evidence

- Give cloud agents small, independent issues.
- Require branch name, allowed files, and verification output.
- Do not accept "works" without commands and output.
- Use reviewer agents on the PR, not in the same implementation thread.

# Swarm Planning

| Stream | Task | Depends on |
|---|---|---|
| Data map | source, variables, caveats | none |
| Estimation | baseline and tests | data map |
| Literature | citations and claim checks | none |
| Review | verification and blockers | PRs |

Use labels: `parallel`, `sequential`, `reviewer`.

# When Cloud Tools Fail

- Record the failure exactly: permission, missing dependency, usage limit, or build error.
- Use local fallback only for the blocked slice.
- Preserve small commits and PR evidence.
- Return to cloud agents on the next independent issue.

# Research Sprint

- Pick one artifact: memo section, test, table, data map, or review log.
- Give the agent one file set and one output.
- Stop after the first useful diff.
- Verify before expanding scope.

**Handoff:** `examples/starter_article/notes/orchestration_log.md`

# Verification Stack

- Unit tests for code paths.
- Replication checks for outputs and tables.
- Referee-style review for identification and overclaiming.
- Playwright or screenshots for browser evidence.
- Human sign-off for interpretation.

**Handoff:** `docs/research_article_harness.md`

# Verification Prompt

```
Act as an economics referee. Review only
examples/card-krueger/docs/research_design_memo.md.

Return blockers first:
- identification;
- data source;
- sample definition;
- outcome units;
- overclaiming.

Do not edit files.
```

# Adoption Checklist

- Choose one real project.
- Add an instruction file and privacy boundary.
- Add one data source map.
- Add one verification command or checklist.
- Create one issue with acceptance criteria.
- Run one reviewer pass before trusting output.

# Close

**Takeaway**

Agentic coding is useful for economists when delegation is bounded by context, acceptance criteria, and verification.

- Start small.
- Keep the human in charge of research claims.
- Make every agent task leave evidence.